

pline-vec

Jasper

May 14, 2025

Contents

1	Introduction	1
2	Examples	2
2.1	changing the main orientation, and sub orientation within it . . .	2
2.2	Parametric surfaces	2
2.3	parametric solids	2
2.4	drawing clipped planes	2
2.5	intersecting planes	4
2.6	math function documentation	4

1 Introduction

I really enjoy making 3D artwork using `TikZ`, especially animations. I really wanted to z-buffer multiple surfaces at once and graph intersecting planes, but these tasks proved to be very difficult using the tools available, namely `pgfplots` and `tikz-3dplot`.

The goal of the package `pline-vec` is to overcome these obstacles. I have already implemented a rigorous z-buffering algorithm that can handle multiple surfaces, and intersecting planes has been made possible, though it lacks automatic z-sorting at this point. That will be fixed.

An additional feature of `pline-vec` is that it defines many useful math functions and operations. For example, there are lua-based `TikZ` functions for rotating objects in 3D, traversing spheres, taking dot products and cross product components, as well as for easily setting multiple `pgfmath` macros at once.

2 Examples

2.1 changing the main orientation, and sub orientation within it

2.2 Parametric surfaces

The code to generate non-intersecting (and also non-tangent) surfaces is complete. The user specifies the domain, range, samples, and **Lua** math function, and pline-vec draws the surface for them. In the future, I will implement the ability to add more keys for things like opacity and color for individual surfaces.

```
\begin{tikzpicture}[/pline-vec/main orientation = {90}{60}{0}]
  \pvappendsurface[
    u min = 0
    ,u max = 720
    ,u samples = 100
    ,v min = 0
    ,v max = 360
    ,v samples = 18
  ]{(2+math.cosd(1.5*u))*math.cosd(u)+0.3*pv_sphere_x(u,1.5*v)}
  {(2+math.cosd(1.5*u))*math.sind(u)+0.3*pv_sphere_y(u,1.5*v)}
  {2*math.sind(1.5*u)+0.3*pv_sphere_z(u,1.5*v)}
  \pvrendersegments
\end{tikzpicture}
```

2.3 parametric solids

I intend to implement code for rendering parametric solids, since the z-buffering technique I use will work for them too.

2.4 drawing clipped planes

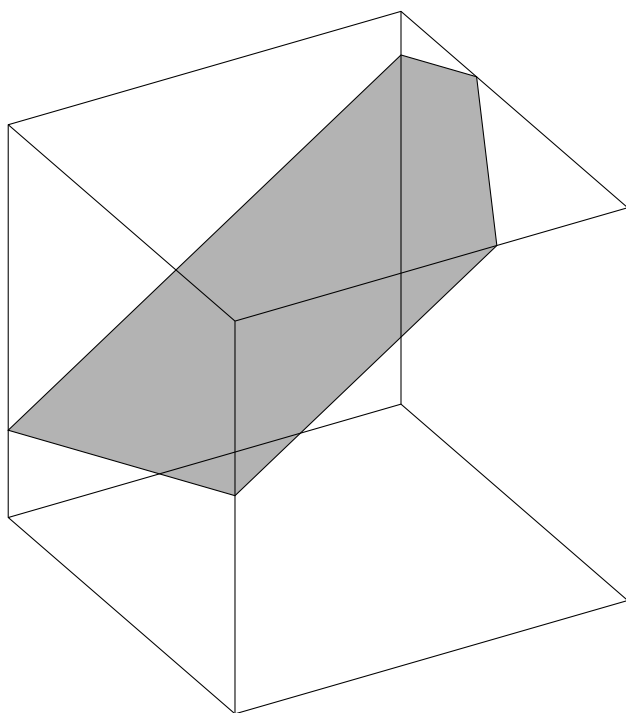
pline-vec lets you draw clipped planes. In the future, this will be done similarly to how it is done for parametric surfaces. For example;

```
\begin{tikzpicture}[/pline-vec/main orientation = {90}{60}{30}]
  \pgfmathsetmacro{\xmin}{-3}
  \pgfmathsetmacro{\xmax}{3}
  \pgfmathsetmacro{\ymin}{-3}
  \pgfmathsetmacro{\ymax}{3}
  \pgfmathsetmacro{\zmin}{-3}
  \pgfmathsetmacro{\zmax}{3}
  % here you set the global options for the coordinate frame
  \tikzset{
    /pline-vec/.cd
    ,xmin = \xmin
    ,xmax = \xmax
```

```

        ,ymin = \ymin
        ,ymax = \ymax
        ,zmin = \zmin
        ,zmax = \zmax
    }
    \pvgetplane[na = 1,nb = 2,nc = 3,nd = 4]{a name}
    % base
    \draw[
        line join = round
        ,canvas is xy plane at z = \zmin
    ] (\xmin,\ymin) rectangle (\xmax,\ymax);
    \draw[
        line join = round
        ,canvas is xy plane at z = \zmax
    ] (\xmin,\ymin) rectangle (\xmax,\ymax);
    \foreach \x/\y
    in {\xmin/\ymin,\xmax/\ymin,\xmax/\ymax,\xmin/\ymax}
    {
        \draw (\x,\y,\zmin) -- (\x,\y,\zmax);
    }
    \draw[
        spath/use = planea name
        ,preaction = {fill,fill opacity = 0.3}
    ];
\end{tikzpicture}

```



2.5 intersecting planes

2.6 math function documentation